Chapter 5

Conditionals and Logic

The programs in the previous chapters do the same thing every time they are run, regardless of the input. For more-complex computations, programs usually react to inputs, check for certain conditions, and generate applicable results. This chapter introduces Java language features for expressing logic and making decisions.

5.1 Relational Operators

Java has six **relational operators** that test the relationship between two values (e.g., whether they are equal, or whether one is greater than the other). The following expressions show how they are used:

```
x == y          // x is equal to y
x != y          // x is not equal to y
x > y           // x is greater than y
x < y           // x is less than y
x >= y          // x is greater than or equal to y
x <= y          // x is less than or equal to y
```

The result of a relational operator is one of two special values: `true` or `false`. These values belong to the data type `boolean`, named after the mathematician George Boole. He developed an algebraic way of representing logic.

You are probably familiar with these operators, but notice how Java is different from mathematical symbols like $=$, $\neq$, and $\geq$. A common error is to use a single `=` instead of a double `==` when comparing values. Remember that `=` is the *assignment* operator, and `==` is a *relational* operator. Also, the operators `=<` and `=>` do not exist.

The two sides of a relational operator have to be compatible. For example, the expression `5 < "6"` is invalid because `5` is an `int` and `"6"` is a `String`. When comparing values of different numeric types, Java applies the same conversion rules you saw previously with the assignment operator. For example, when evaluating the expression `5 < 6.0`, Java automatically converts the `5` to `5.0`.

5.2 The if-else Statement

To write useful programs, we almost always need to check conditions and react accordingly. **Conditional statements** give us this ability. The simplest conditional statement in Java is the `if` statement:

```
if (x > 0) {
    System.out.println("x is positive");
}
```

The expression in parentheses is called the *condition*. If it is true, the statements in braces get executed. If the condition is false, execution skips over that **block** of code. The condition in parentheses can be any `boolean` expression.

A second form of conditional statement has two possibilities, indicated by `if` and `else`. The possibilities are called **branches**, and the condition determines which branch gets executed:

```
if (x % 2 == 0) {
    System.out.println("x is even");
} else {
    System.out.println("x is odd");
}
```

If the remainder when `x` is divided by 2 is 0, we know that `x` is even, and the program displays a message to that effect. If the condition is false, the second

print statement is executed instead. Since the condition must be true or false, exactly one of the branches will run.

The braces are optional for branches that have only one statement. So we could have written the previous example this way:

```
if (x % 2 == 0)
    System.out.println("x is even");
else
    System.out.println("x is odd");
```

However, it's better to use braces—even when they are optional—to avoid making the mistake of adding statements to a one-line `if` or `else` block. This code is misleading because it's not indented correctly:

```
if (x > 0)
    System.out.println("x is positive");
    System.out.println("x is not zero");
```

Since there are no braces, only the first `println` is part of the `if` statement. Here is what the compiler actually sees:

```
if (x > 0) {
    System.out.println("x is positive");
}
    System.out.println("x is not zero");
```

As a result, the second `println` runs no matter what. Even experienced programmers make this mistake; search the web for Apple's "goto fail" bug.

In all previous examples, notice that there is no semicolon at the end of the `if` or `else` lines. Instead, a new block should be defined using braces. Another common mistake is to put a semicolon after the condition, like this:

```
int x = 1;
if (x % 2 == 0); {  // incorrect semicolon
    System.out.println("x is even");
}
```

This code will compile, but the program will output `"x is even"` regardless of the value of `x`. Here is the same incorrect code with better formatting:

```
int x = 1;
if (x % 2 == 0)
    ;  // empty statement
{
    System.out.println("x is even");
}
```

Because of the semicolon, the `if` statement compiles as if there are no braces, and the subsequent block runs independently. As a general rule, each line of Java code should end with a semicolon or brace—but not both.

The compiler won't complain if you omit optional braces or write empty statements. Doing so is allowed by the Java language, but it often results in bugs that are difficult to find. Development tools like Checkstyle (see Appendix **??**) can warn you about these and other kinds of programming mistakes.

5.3 Chaining and Nesting

Sometimes you want to check related conditions and choose one of several actions. One way to do this is by **chaining** a series of `if` and `else` blocks:

```
if (x > 0) {
    System.out.println("x is positive");
} else if (x < 0) {
    System.out.println("x is negative");
} else {
    System.out.println("x is zero");
}
```

These chains can be as long as you want, although they can be difficult to read if they get out of hand. One way to make them easier to read is to use standard indentation, as demonstrated in these examples. If you keep all the statements and braces lined up, you are less likely to make syntax errors.

Notice that the last branch is simply `else`, not `else if (x == 0)`. At this point in the chain, we know that `x` is not positive and `x` is not negative. There is no need to test whether `x` is 0, because there is no other possibility.

In addition to chaining, you can also make complex decisions by **nesting** one conditional statement inside another. We could have written the previous example as follows:

```
if (x > 0) {
    System.out.println("x is positive");
} else {
    if (x < 0) {
        System.out.println("x is negative");
    } else {
        System.out.println("x is zero");
    }
}
```

The outer conditional has two branches. The first branch contains a print statement, and the second branch contains another conditional statement, which has two branches of its own. These two branches are also print statements, but they could have been conditional statements as well.

These kinds of nested structures are common, but they can become difficult to read very quickly. Good indentation is essential to make the structure (or intended structure) apparent to the reader.

5.4 The switch Statement

If you need to make a series of decisions, chaining `else if` blocks can get long and redundant. For example, consider a program that converts integers like 1, 2, and 3 into words like `"one"`, `"two"`, and `"three"`:

```
if (number == 1) {
    word = "one";
} else if (number == 2) {
    word = "two";
} else if (number == 3) {
    word = "three";
} else {
    word = "unknown";
}
```

This chain could go on and on, especially for banking programs that write numbers in long form (e.g., "one hundred twenty-three and 45/100 dollars"). An alternative way to evaluate many possible values of an expression is to use a `switch` statement:

```
switch (number) {
    case 1:
        word = "one";
        break;
    case 2:
        word = "two";
        break;
    case 3:
        word = "three";
        break;
    default:
        word = "unknown";
        break;
}
```

The body of a `switch` statement is organized into one or more `case` blocks. Each `case` ends with a `break` statement, which exits the `switch` body. The `default` block is optional and executed only if none of the cases apply.

Although `switch` statements appear longer than chained `else if` blocks, they are particularly useful when multiple cases can be grouped:

```
switch (food) {
    case "apple":
    case "banana":
    case "cherry":
        System.out.println("Fruit!");
        break;
    case "asparagus":
    case "broccoli":
    case "carrot":
        System.out.println("Vegetable!");
        break;
}
```

5.5 Logical Operators

In addition to the relational operators, Java also has three **logical operators**: `&&`, `||`, and `!`, which respectively stand for *and*, *or*, and *not*. The results of these operators are similar to their meanings in English. For example:

- `x > 0 && x < 10` is true when `x` is greater than 0 *and* less than 10.
- `x < 0 || x > 10` is true if either condition is true; that is, if `x` is less than 0 *or* greater than 10.
- `!(x > 0)` is true if `x` is *not* greater than 0. The parentheses are necessary in this example because, in the order of operations, `!` comes before `>`.

In order for an expression with `&&` to be true, both sides of the `&&` operator must be true. And in order for an expression with `||` to be false, both sides of the `||` operator must be false.

The `&&` operator can be used to simplify nested `if` statements. For example, the following code can be rewritten with a single condition:

```
if (x == 0) {
    if (y == 0) {
        System.out.println("Both x and y are zero");
    }
}
```

```
// combined
if (x == 0 && y == 0) {
    System.out.println("Both x and y are zero");
}
```

Likewise, the `||` operator can simplify chained `if` statements. Since the branches are the same, there is no need to duplicate that code:

```
if (x == 0) {
    System.out.println("Either x or y is zero");
} else if (y == 0) {
    System.out.println("Either x or y is zero");
}
```

```
// combined
if (x == 0 || y == 0) {
    System.out.println("Either x or y is zero");
}
```

Then again, if the statements in the branches were different, we could not combine them into one block. But it's useful to explore different ways of representing the same logic, especially when it's complex.

Logical operators evaluate the second expression *only when necessary*. For example, `true || anything` is always true, so Java does not need to evaluate the expression `anything`. Likewise, `false && anything` is always false.

Ignoring the second operand, when possible, is called **short-circuit** evaluation, by analogy with an electrical circuit. Short-circuit evaluation can save time, especially if `anything` takes a long time to evaluate. It can also avoid unnecessary errors, if `anything` might fail.

5.6 De Morgan's Laws

Sometimes you need to negate an expression containing a mix of relational and logical operators. For example, to test if `x` and `y` are both nonzero, you could write the following:

```
if (!(x == 0 || y == 0)) {
    System.out.println("Neither x nor y is zero");
}
```

This condition is difficult to read because of the `!` and parentheses. A better way to negate logic expressions is to apply **De Morgan's laws**:

- `!(A && B)` is the same as `!A || !B`
- `!(A || B)` is the same as `!A && !B`

In words, negating a logical expression is the same as negating each term and changing the operator. The `!` operator takes precedence over `&&` and `||`, so you don't have to put parentheses around the individual terms `!A` and `!B`.

De Morgan's laws also apply to the relational operators. In this case, negating each term means using the "opposite" relational operator:

- `!(x < 5 && y == 3)` is the same as `x >= 5 || y != 3`
- `!(x >= 1 || y != 7)` is the same as `x < 1 && y == 7`

It may help to read these examples out loud in English. For instance, "If I don't want the case where x is less than 5 and y is 3, then I need x to be greater than or equal to 5, or I need y to be anything but 3."

Returning to the previous example, here is the revised condition. In English, it reads, "If x is not zero and y is not zero." The logic is the same, and the source code is easier to read:

```
if (x != 0 && y != 0) {
    System.out.println("Neither x nor y is zero");
}
```

5.7 Boolean Variables

To store a `true` or `false` value, you need a `boolean` variable. You can declare and assign them like other variables. In this example, the first line is a variable declaration, the second is an assignment, and the third is both:

```
boolean flag;
flag = true;
boolean testResult = false;
```

Since relational and logical operators evaluate to a `boolean` value, you can store the result of a comparison in a variable:

```
boolean evenFlag = (x % 2 == 0);     // true if x is even
boolean positiveFlag = (x > 0);      // true if x is positive
```

The parentheses are unnecessary, but they make the code easier to understand. A variable defined in this way is called a **flag**, because it signals, or "flags", the presence or absence of a condition.

You can use flag variables as part of a conditional statement:

```
if (evenFlag) {
    System.out.println("n was even when I checked it");
}
```

Flags may not seem that useful at this point, but they will help simplify complex conditions later. Each part of a condition can be stored in a separate flag, and these flags can be combined with logical operators.

Notice that we didn't have to write `if (evenFlag == true)`. Since `evenFlag` is a `boolean`, it's already a condition. To check if a flag is `false`, we simply negate the flag:

```
if (!evenFlag) {
    System.out.println("n was odd when I checked it");
}
```

In general, you should never compare anything to `true` or `false`. Doing so makes the code more verbose and awkward to read out loud.

5.8 Boolean Methods

Methods can return `boolean` values, just like any other type, which is often convenient for hiding tests inside methods. For example:

```
public static boolean isSingleDigit(int x) {
    if (x > -10 && x < 10) {
        return true;
    } else {
        return false;
    }
}
```

The name of this method is `isSingleDigit`. It is common to give `boolean` methods names that sound like yes/no questions. Since the return type is `boolean`, the return statement has to provide a boolean expression.

The code itself is straightforward, although it is longer than it needs to be. Remember that the expression `x > -10 && x < 10` has type `boolean`, so there is nothing wrong with returning it directly (without the `if` statement):

```
public static boolean isSingleDigit(int x) {
    return x > -10 && x < 10;
}
```

In `main`, you can invoke the method in the usual ways:

```
System.out.println(isSingleDigit(2));
boolean bigFlag = !isSingleDigit(17);
```

The first line displays `true` because 2 is a single-digit number. The second line sets `bigFlag` to `true`, because 17 is *not* a single-digit number.

Conditional statements often invoke `boolean` methods and use the result as the condition:

```
if (isSingleDigit(z)) {
    System.out.println("z is small");
} else {
    System.out.println("z is big");
}
```

Examples like this one almost read like English: "If is single digit z, print z is small else print z is big."

5.9 Validating Input

One of the most important tasks in any computer program is to **validate** input from the user. People often make mistakes while typing, especially on smartphones, and incorrect inputs may cause your program to fail.

Even worse, someone (i.e., a **hacker**) may intentionally try to break into your system by entering unexpected inputs. You should never assume that users will input the right kind of data.

Consider this simple program that prompts the user for a number and computes its logarithm:

```
Scanner in = new Scanner(System.in);
System.out.print("Enter a number: ");
double x = in.nextDouble();
double y = Math.log(x);
System.out.println("The log is " + y);
```

In mathematics, the natural logarithm (base e) is undefined when $x \leq 0$. In Java, if you ask for `Math.log(-1)`, it returns **NaN**, which stands for "not a number". We can check for this condition and print an appropriate message:

```
if (x > 0) {
    double y = Math.log(x);
    System.out.println("The log is " + y);
} else {
    System.out.println("The log is undefined");
}
```

The output is better now, but there is another problem. What if the user doesn't enter a number at all? What would happen if they typed the word "hello", either by accident or on purpose?

```
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Scanner.java:864)
    at java.util.Scanner.next(Scanner.java:1485)
    at java.util.Scanner.nextDouble(Scanner.java:2413)
    at Logarithm.main(Logarithm.java:8)
```

If the user inputs a `String` when we expect a `double`, Java reports an "input mismatch" exception. We can prevent this run-time error from happening by testing the input first.

The `Scanner` class provides `hasNextDouble`, which checks whether the next input can be interpreted as a `double`. If not, we can display an error message:

```
if (!in.hasNextDouble()) {
    String word = in.next();
    System.err.println(word + "is not a number");
}
```

In contrast to `in.nextLine`, which returns an entire line of input, the `in.next` method returns only the next token of input. We can use `in.next` to show the user exactly which word they typed was not a number.

This example also uses `System.err`, which is an `OutputStream` for error messages and warnings. Some development environments display output to `System.err` with a different color or in a separate window.

5.10 Example Program

In this chapter, you have seen relational and logical operators, `if` statements, boolean methods, and validating input. The following program shows how the individual code examples in the previous section fit together:

```
import java.util.Scanner;

/**
 * Demonstrates input validation using if statements.
 */
public class Logarithm {

    public static void main(String[] args) {

        // prompt for input
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a number: ");

        // check the format
        if (!in.hasNextDouble()) {
            String word = in.next();
            System.err.println(word + " is not a number");
            return;
        }

        // check the range
        double x = in.nextDouble();
        if (x > 0) {
            double y = Math.log(x);
            System.out.println("The log is " + y);
        } else {
            System.out.println("The log is undefined");
        }
    }
}
```

Notice that the `return` statement allows you to exit a method before you reach the end of it. Returning from `main` terminates the program.

What started as five lines of code at the beginning of Section 5.9 is now a 30-line program. Making programs robust (and secure) often requires a lot of additional checking, as shown in this example.

It's important to write comments every few lines to make your code easier to understand. Comments not only help other people read your code, but also help you document what you're trying to do. If there's a mistake in the code, finding it will be a lot easier when there are good comments.

5.11 Vocabulary

boolean: A data type with only two possible values, `true` and `false`.

relational operator: An operator that compares two values and produces a `boolean` indicating the relationship between them.

conditional statement: A statement that uses a condition to determine which statements to execute.

block: A sequence of statements, surrounded by braces, that generally runs as the result of a condition.

branch: One of the alternative blocks after a conditional statement. For example, an `if-else` statement has two branches.

chaining: A way of joining several conditional statements in sequence.

nesting: Putting a conditional statement inside one or both branches of another conditional statement.

logical operator: An operator that combines boolean values and produces a boolean value.

short circuit: A way of evaluating logical operators that evaluates the second operand only if necessary.

De Morgan's laws: Mathematical rules that show how to negate a logical expression.

flag: A variable (usually `boolean`) that represents a condition or status.

validate: To confirm that an input value is of the correct type and within the expected range.

hacker: A programmer who breaks into computer systems. The term hacker may also apply to someone who enjoys writing code.

NaN: A special floating-point value that stands for "not a number".

5.12 Exercises

The code for this chapter is in the *ch05* directory of *ThinkJavaCode2*. See page x for instructions on how to download the repository. Before you start the exercises, we recommend that you compile and run the examples.

If you have not already read Appendix **??**, now might be a good time. It describes Checkstyle, a tool that analyzes many aspects of your source code.

Exercise 5.1 Rewrite the following code by using a single `if` statement:

```
if (x > 0) {
    if (x < 10) {
        System.out.println("positive single digit number.");
    }
}
```

Exercise 5.2 Now that we have conditional statements, we can get back to the *Guess My Number* game from Exercise 3.4.

You should already have a program that chooses a random number, prompts the user to guess it, and displays the difference between the guess and the chosen number.

By adding a small amount of code at a time and testing as you go, modify the program so it tells the user whether the guess is too high or too low, and then prompts the user for another guess.

The program should continue until the user gets it right or guesses incorrectly three times. If the user guesses the correct number, display a message and terminate the program.

Exercise 5.3 Fermat's Last Theorem says that there are no integers a, b, c, and n such that $a^n + b^n = c^n$, except when $n \leq 2$.

Write a program named *Fermat.java* that inputs four integers (`a`, `b`, `c`, and `n`) and checks to see if Fermat's theorem holds. If n is greater than 2 and $a^n + b^n = c^n$, the program should display "Holy smokes, Fermat was wrong!" Otherwise, the program should display "No, that doesn't work."

Hint: You might want to use `Math.pow`.

Exercise 5.4 Using the following variables, evaluate the logic expressions in the table that follows. Write your answers as true, false, or error.

```
boolean yes = true;
boolean no = false;
int loVal = -999;
int hiVal = 999;
double grade = 87.5;
double amount = 50.0;
String hello = "world";
```

Expression	Result
`yes == no || grade > amount`	
`amount == 40.0 || 50.0`	
`hiVal != loVal || loVal < 0`	
`True || hello.length() > 0`	
`hello.isEmpty() & & yes`	
`grade <= 100 & & !false`	
`!yes || no`	
`grade > 75 > amount`	
`amount <= hiVal & & amount >= loVal`	
`no & & !no || yes & & !yes`	

Exercise 5.5 What is the output of the following program? Determine the answer without using a computer.

```
public static void main(String[] args) {
    boolean flag1 = isHoopy(202);
    boolean flag2 = isFrabjuous(202);
    System.out.println(flag1);
    System.out.println(flag2);
    if (flag1 && flag2) {
        System.out.println("ping!");
    }
    if (flag1 || flag2) {
        System.out.println("pong!");
    }
}
```

```
public static boolean isHoopy(int x) {
    boolean hoopyFlag;
    if (x % 2 == 0) {
        hoopyFlag = true;
    } else {
        hoopyFlag = false;
    }
    return hoopyFlag;
}
```

```
public static boolean isFrabjuous(int x) {
    boolean frabjuousFlag;
    if (x > 0) {
        frabjuousFlag = true;
    } else {
        frabjuousFlag = false;
    }
    return frabjuousFlag;
}
```

The purpose of this exercise is to make sure you understand logical operators and the flow of execution through methods.

Exercise 5.6 Write a program named *Quadratic.java* that finds the roots of $ax^2 + bx + c = 0$ using the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Prompt the user to input integers for a, b, and c. Compute the two solutions for x, and display the results.

Your program should be able to handle inputs for which there is only one or no solution. Specifically, it should not divide by zero or take the square root of a negative number.

Be sure to validate all inputs. The user should never see an input mismatch exception. Display specific error messages that include the invalid input.

Exercise 5.7 If you are given three sticks, you may or may not be able to arrange them in a triangle. For example, if one of the sticks is 12 inches long and the other two are 1 inch long, you will not be able to get the short sticks to meet in the middle. For any three lengths, there is a simple test to see if it is possible to form a triangle:

> If any of the three lengths is greater than the sum of the other two, you cannot form a triangle.

Write a program named *Triangle.java* that inputs three integers, and then outputs whether you can (or cannot) form a triangle from the given lengths. Reuse your code from the previous exercise to validate the inputs. Display an error if any of the lengths are negative or zero.